

UMassAmherst | Manning College of Information & Computer Sciences

Programming Methodology

Lab 5: Midterm 1 Review

Wednesday, October 7th, 2026

Weekly Lab Agenda

- Go over reminders/goals
- Review past material
- Work in groups of 3-4 to solve a few exercises.
  - Please sit with your group from last week.
- Discussion leaders will walk around and answer questions.
- Solutions to exercises will be reviewed as a class.

Reminders

- **Midterm 1 is TONIGHT (see next slide for details)**
- Homework 5 is due next Wednesday (10/14) at 11:59 pm.
  - Come to office hours for help!
- If you need to miss lab for a valid and documented reason:
  - Fill out the excusal form on Canvas before the lab.
  - Waking up late or the bus being late are not excusable reasons.
- **Today, your lab grade is based solely on attendance**
  - There is no starter code
  - **You must stay the entire time to receive credit**

Exam Logistics

- **Time: Today (10/7) from 7-9 pm**
- **Location:**
  - **Section 1 (Perretta): Computer Science Laboratories E110**
  - **Section 2 (Minea): Furcolo Hall 125**

Exam Advice

- **Work on the questions in any order**; move on to another if you're stuck.
- **Don't leave any question blank**.
  - Write down notes of what you know to jog your memory.
  - Please don't write irrelevant comments, which hinder grading.
- **Read each question carefully** and mark it up for your understanding.
  - Underline important parts.
  - What does the function take in and return? Expected behavior?
- **If you have a question, raise your hand** and a proctor will come over.
  - Please keep your voice down when asking questions.
- **Use a pencil**, not a pen.
  - If you don’t have any, you can get them for free at the library!

Today's Goal

- Midterm Review

Topic Poll

- We will do a few exercises on your preferred topics
- Raise your hand for your top 2-3 topics
  - Type signatures
  - Higher-order functions
  - Mental models + Closures
  - Lists
  - Testing

Type Signatures

Exercise: Type Signatures

What are the types of **g**, **a**, and **x**?

```
const g = (a, x) => a.map(f => (x => f(f(x)))).map(f => f(x));
```

- Look at the smallest atomic parts of this function.
- Write down what you know immediately.
- Deduce the rest using a combination of what you figured out previously.

Solution: Type Signatures

```
const g = (a, x) => a.map(f => (x => f(f(x)))).map(f => f(x));
```

- **a** is an **Array** of some generic type (like **U[]**)
- **.map** iterates over all elements of **a**
  - The **f**'s in **.map(f => (x => f(f(x))))** are the elements of **a**
  - They are being called, so they are functions
  - The calls are nested, so they must take in the same type they return
  - **a: ((x: T) => T)[] x: T**
- **a.map(f => (x => f(f(x))))** returns array of type **((x: T) => T)[]**
- **.map(f => f(x))** returns array with results of calling new functions
  - Functions return **T**, so this returns **T[]**
- Answer: **g<T>(a: ((x: T) => T)[], x: T): T[]**

Higher-Order Functions

Exercise: HOFs

```
function reducer(acc, e) {
  return {
    x: acc.y + e,
    y: acc.x
  };
}

const pair = a => a.reduce(reducer, {x: "", y: ""});
```

What is **pair(["that", "is", "a", "short", "text"])** ?

Solution: HOFs

```
function reducer(acc, e) { return { x: acc.y + e, y: acc.x }; }
```

```
a = ["that", "is", "a", "short", "text"]
```

index in **a**	**e**	**acc**
before looping	-	{x:"",y:""}
0	"that"	{ x:"that", y:"" }
1	"is"	{ x:"is", y:"that" }
2	"a"	{x:"thata", y:"is"}
3	"short"	{x:"isshort", y:"thata"}
4	"text"	{x:"thatatext",y:"isshort"}

Intuition:

reducer *switches* the values of **x** and **y** and *concatenates* the current element **e** onto the value of **x**

Mental Models + Closures

Exercise: Mental Models

For the line defining o1 below, state how many objects except { } are created when executing that line. State what values are printed. Explain your answers.

```
1   function f1(k) {
2     let o = {};
3     while (--k >= 0) {
4       o = {val: () => k, next: o}
5     }
6     return o;
7   }
8
9   let o1 = f1(3);
10  console.log(o1.val());
11  console.log(o1.next.val());
```

Solution: Mental Models

For the line defining **o1** below, state how many objects except **{ }** are created when executing that line. State what values are printed. Explain your answers.

```
1   function f1(k) {
2     let o = {};
3     while (--k >= 0) {
4       o = {val: () => k, next: o}
5     }
6     return o;
7   }
8
9   let o1 = f1(3);
10  console.log(o1.val());
11  console.log(o1.next.val());
```

The loop at line 3 executes three times.

- Three objects are created and linked.
- **next** references the previous created object.
- The three **val** closures share the same environment and refer to the same variable **k**; they are identical.

Calling **val** evaluates **k**.

- **f1** has completed and **k** is **-1**.
- **-1** is printed.

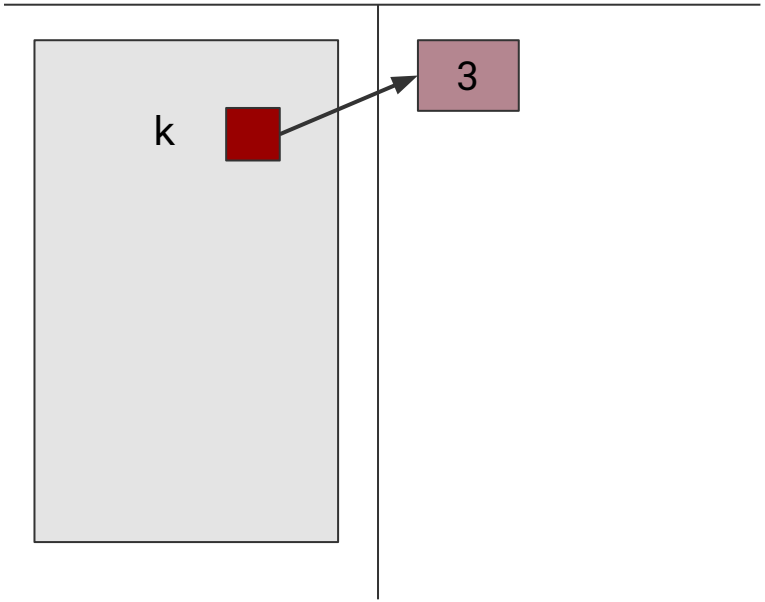
o1.next.val is an identical closure **() => k**.

- This prints **-1**, the value of **k**.

Solution: Mental Models

When **f1** starts executing, **k = 3** is in the value map.

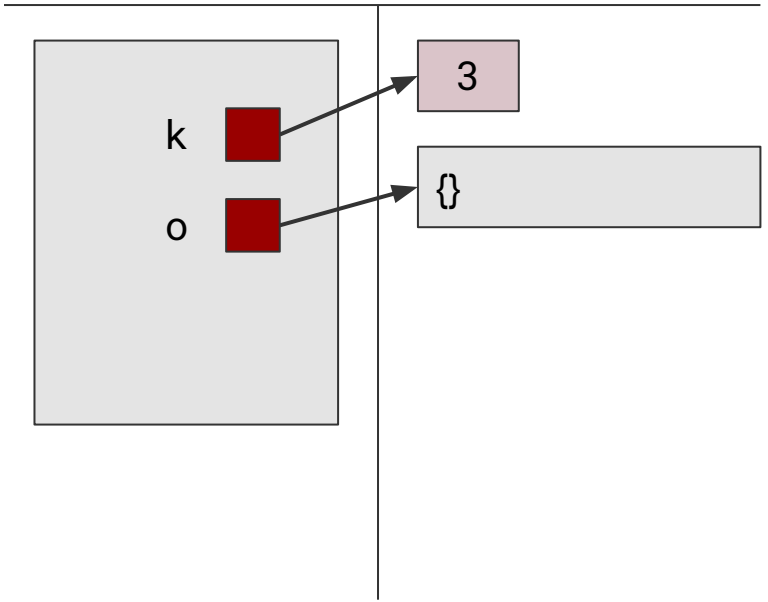
```
1   function f1(k) {
2     let o = {};
3     while (--k >= 0) {
4       o = {val: () => k, next: o}
5     }
6     return o;
7   }
8
9   let o1 = f1(3);
10  console.log(o1.val());
11  console.log(o1.next.val());
```



Solution: Mental Models

Line 2 creates an empty object in memory and stores its reference in **o**.

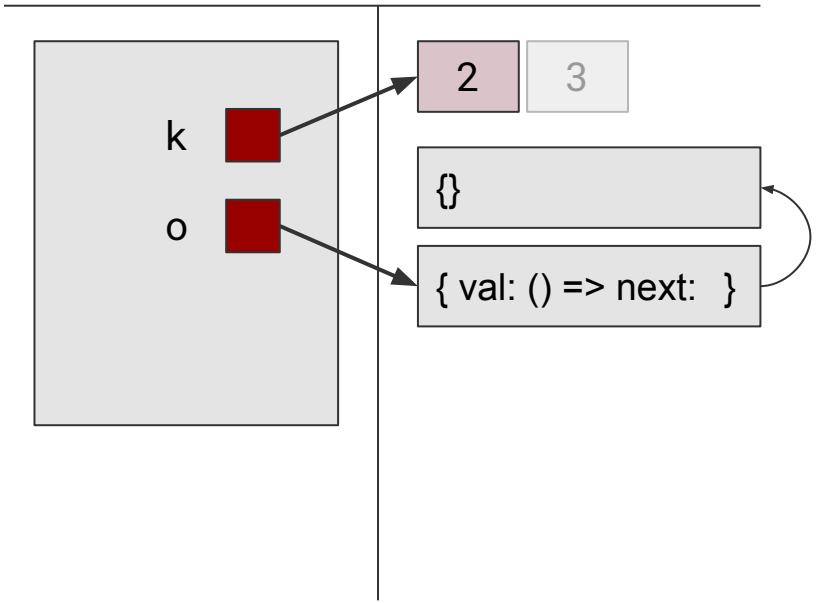
```
1   function f1(k) {
2     let o = {};
3     while (--k >= 0) {
4       o = {val: () => k, next: o}
5     }
6     return o;
7   }
8
9   let o1 = f1(3);
10  console.log(o1.val());
11  console.log(o1.next.val());
```



Solution: Mental Models

First iteration of loop decreases value of **k** by **1** and creates a new object; **next** field of newly created object copies reference stored in **o**; **o** stores reference to newly created object.

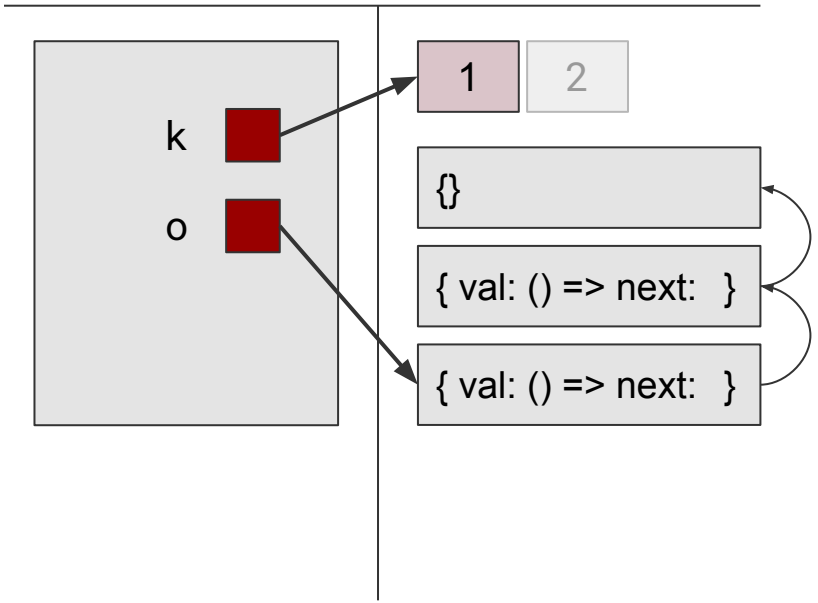
```
1   function f1(k) {
2     let o = {};
3     while (--k >= 0) {
4       o = {val: () => k, next: o}
5     }
6     return o;
7   }
8
9   let o1 = f1(3);
10  console.log(o1.val());
11  console.log(o1.next.val());
```



Solution: Mental Models

Second iteration of the loop, another object is created.

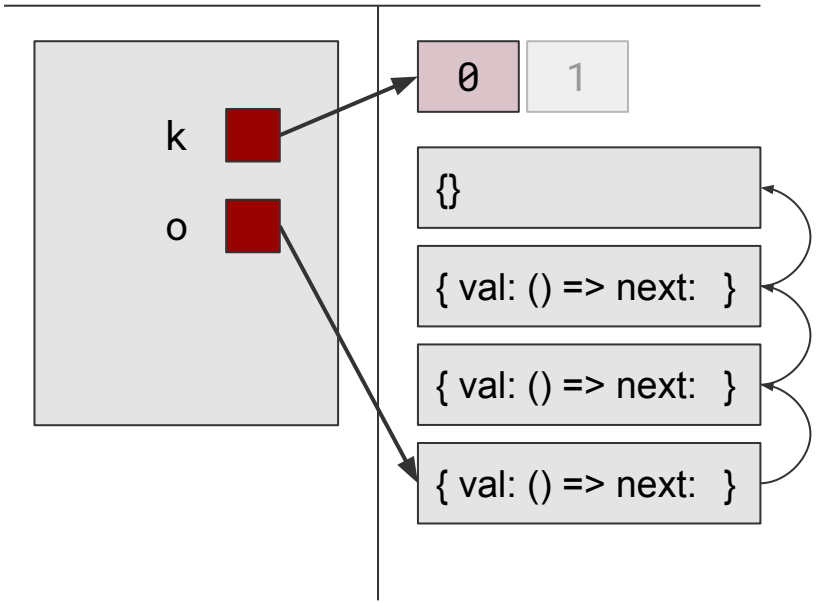
```
1   function f1(k) {
2     let o = {};
3     while (--k >= 0) {
4       o = {val: () => k, next: o}
5     }
6     return o;
7   }
8
9   let o1 = f1(3);
10  console.log(o1.val());
11  console.log(o1.next.val());
```



Solution: Mental Models

Third iteration of the loop. One more object created.

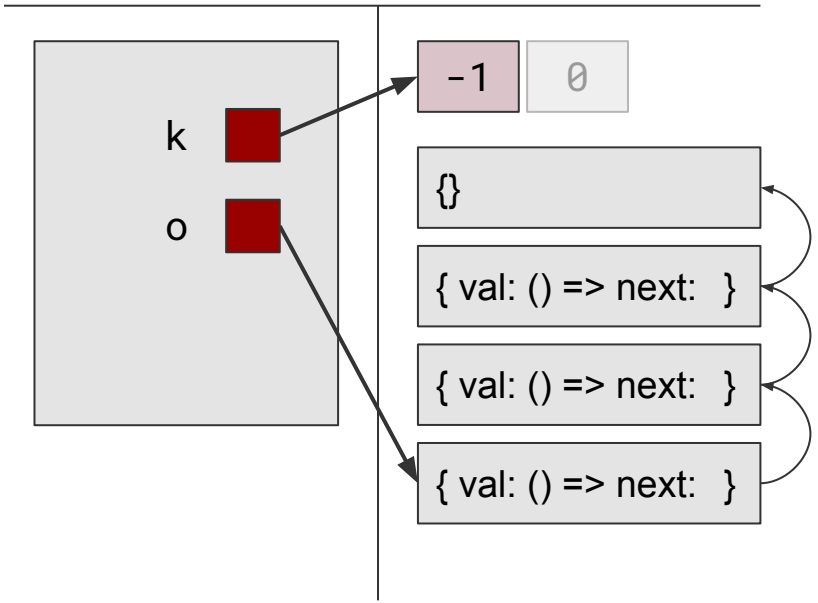
```
1   function f1(k) {
2     let o = {};
3     while (--k >= 0) {
4       o = {val: () => k, next: o}
5     }
6     return o;
7   }
8
9   let o1 = f1(3);
10  console.log(o1.val());
11  console.log(o1.next.val());
```



Solution: Mental Models

The **while** condition is evaluated, which decreases **k** by **1**.
The body of the loop isn't executed.

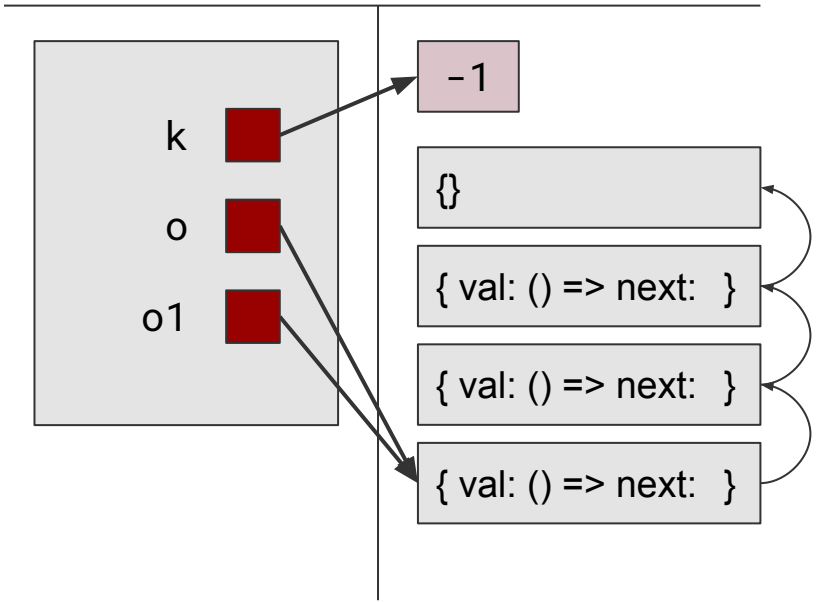
```
1   function f1(k) {
2     let o = {};
3     while (--k >= 0) {
4       o = {val: () => k, next: o}
5     }
6     return o;
7   }
8
9   let o1 = f1(3);
10  console.log(o1.val());
11  console.log(o1.next.val());
```



Solution: Mental Models

Upon return, **o1** will have the same reference value returned through **o**.

```
1   function f1(k) {
2     let o = {};
3     while (--k >= 0) {
4       o = {val: () => k, next: o}
5     }
6     return o;
7   }
8
9   let o1 = f1(3);
10  console.log(o1.val());
11  console.log(o1.next.val());
```



Lists

Exercise: Lists

Write a function

```
sumEveryK(lst: List<number>, k: number): List<number>
```

that does the following:

- Splits **`lst`** into groups of **`k`** adjacent elements (last group may have fewer)
- Returns a list with the sums of each group (in the same order)

Assume **`k`** is a positive natural. Try not to create other temporary lists.

Example:

For **`lst = 1,0,3,5,2,3,7`** with **`k = 3`**,

Output of **`sumEveryK(lst, k)`** should be **`list = 4,10,7`**

Solution: Lists (with HOFs)

```
function sumEveryK(lst: List<number>, k: number): List<number> {
  let sum = 0, rem = k;
  const result = lst.reduce((acc, elem) => {
    let output = acc;
    if (rem === 0) {
      output = node(sum, acc);
      sum = 0, rem = k;
    }
    rem--;
    sum += elem;
    return output;
  }, empty<number>());
  return reverseList(node(sum, result));
}
```

Solution: Lists (with Recursion)

```
function sumEveryK(lst: List<number>, k: number): List<number> {
  // rem: count left to make group of k, acc: accumulated sum
  function sumRec(lst: List<number>, rem: number,acc: number)
      : List<number> {
    // add last sum
    if (lst.isEmpty()) return rem === k ? lst : node(acc, lst);
    // set up recursion
    const newS = acc + lst.head()
    const tl = lst.tail();
    if (--rem === 0) return node(newS, sumRec(tl, k, 0));
    return sumRec(tl, rem, newS);
  }
  return sumRec(lst, k, 0);
}
```

Testing

Exercise: Testing

The **findIndex** function returns the index of the first element of array **a** for which **f** returns **true**, or **-1** if no such element exists.

```
function findIndex<T>(a:T[], f: (x: T) => boolean): number {
  for (let i = 0; i < a.length; ++i) {
    if (f(a[i])) return i;
  }
  return -1;
}
```

Write three representative cases that you would use to test **findIndex**.

You do not need to write code, but do clearly indicate

- inputs
- output
- purpose of test

Solution: Testing

```
function findIndex<T>(a:T[], f: (x: T) => boolean): number {
  for (let i = 0; i < a.length; ++i) {
    if (f(a[i])) return i;
  }
  return -1;
}
```

Example tests:

Input **a**	Input **f**	Output	Purpose
`[]`	`x => x > 0`	`-1`	No element is found for an empty array
`[4, 3, 2, 5]`	`x => x % 2 > 0`	`1`	First index is returned for multiple matches
`['hi', 'ho']`	`s => s.length > 2`	`-1`	No element matches

GOOD LUCK!!!



🤠